

Modul 2: Backpropagation dan Automatic Differentiation

Nama: Natasya Amavisca **NIM:** 123450024 **Kelas:** Deep Learning RB

Tanggal: 2026-09-22

Simpan berkas ini sebagai `M02_NIM.ipynb` sebelum mulai mengerjakan.

Petunjuk

1. Ganti seluruh penanda `TODO` . Jangan menghapus sel pemeriksaan.
2. Nilai Kasus 1 tidak boleh diubah; seluruh angka pada modul mengacu padanya.
3. Gunakan `float64` untuk semua perhitungan gradien.
4. Tuliskan turunan manual pada sel markdown, bukan hanya di kertas.
5. Notebook harus lolos *Restart Kernel and Run All* sebelum dikumpulkan.
6. Luaran: `M02_NIM.ipynb` , `M02_NIM.pdf` , dan `M02_NIM_metrics.csv` .

```
In [1]: import platform
import random

import numpy as np
import pandas as pd
import torch
from torch import nn

NIM = '123450024' # contoh: '120450123'
SEED = int(str(NIM)[-4:]) if str(NIM).isdigit() else 42 # seed individual
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

def seed_everything(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

seed_everything(SEED)
torch.set_default_dtype(torch.float64)
pd.set_option('display.precision', 8)
print({'python': platform.python_version(), 'numpy': np.__version__,
      'torch': torch.__version__, 'device': str(DEVICE), 'seed': SEED})

{'python': '3.12.5', 'numpy': '2.0.1', 'torch': '2.6.0+cpu', 'device': 'cpu', 'seed': 24}
```

A. Pre-lab - 10 poin

Jawab sebelum sesi praktikum dimulai.

1. **Gradien lokal vs gradien total pada satu simpul:** Gradien lokal adalah turunan suatu operasi terhadap inputnya secara langsung pada satu simpul. Sedangkan gradien total adalah pengaruh suatu simpul terhadap loss setelah memperhitungkan seluruh jalur dari simpul tersebut menuju loss melalui chain rule.
2. **Mengapa `backward()` hanya dapat dipanggil pada tensor skalar:** Secara default, `backward()` menghitung gradien dari satu nilai output (skalar) terhadap tensor yang memiliki `requires_grad=True`. Jika output bukan skalar, PyTorch membutuhkan `gradient/upstream gradient` tambahan untuk menentukan kombinasi vector-Jacobian product yang ingin dihitung. Karena loss biasanya berupa skalar, `loss.backward()` dapat dipanggil langsung tanpa argumen tambahan.
3. **Isi `.grad` bila `backward()` dipanggil dua kali tanpa `zero_grad()`:** PyTorch melakukan **akumulasi gradien** pada atribut `.grad`, bukan mengganti nilai gradien sebelumnya. Jika gradien dari satu kali `backward()` adalah g , maka ketika `backward()` dipanggil dua kali tanpa `zero_grad()`, nilai `.grad` menjadi: $g + g = 2g$. Jadi, gradien sebelumnya akan ditambahkan dengan gradien baru. Oleh karena itu, gradien perlu dikosongkan terlebih dahulu sebelum melakukan `backward()` berikutnya, misalnya dengan: `optimizer.zero_grad()`
4. **Mengapa turunan BCE-with-logits terhadap logit berbentuk $-p \ln p - (1-p) \ln (1-p)$, bukan $-y \ln p - (1-y) \ln (1-p)$:** BCE memiliki bentuk:

$$L = -[y \log(p) + (1-y) \log(1-p)]$$

Turunan terhadap probabilitas p adalah:

$$\frac{\partial L}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}$$

Karena:

$$p = \sigma(z)$$

maka turunan sigmoid terhadap logit z adalah:

$$\frac{\partial p}{\partial z} = p(1-p)$$

Dengan menggunakan **chain rule**:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial p} \frac{\partial p}{\partial z}$$

sehingga:

$$\frac{\partial L}{\partial z} = \left(-\frac{y}{p} + \frac{1-y}{1-p} \right) p(1-p)$$

yang dapat disederhanakan menjadi:

$$\frac{\partial L}{\partial z} = p - y$$

Jadi, $-y/p$ bukan turunan BCE terhadap logit z , melainkan hanya salah satu bagian dari turunan loss terhadap probabilitas p .

Graf komputasi Kasus 1. Tuliskan urutan simpul dari $\mathbf{x}$ sampai $\mathcal{L}$, lalu tandai gradien lokal di setiap simpul:

$$\mathbf{x} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{h} \rightarrow \mathbf{z}^{(2)} \rightarrow \mathcal{L}$$

dengan gradien lokal:

- $\mathbf{z}^{(1)} = W_1 \mathbf{x} + b_1 \rightarrow \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{x}} = W_1$
- $\mathbf{h} = \text{ReLU}(\mathbf{z}^{(1)}) \rightarrow \text{ReLU}'(\mathbf{z}^{(1)})$
- $\mathbf{z}^{(2)} = W_2 \mathbf{h} + b_2 \rightarrow \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}} = W_2$
- $\mathcal{L} = \text{BCEWithLogits}(\mathbf{z}^{(2)}, y) \rightarrow \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} = p - y$

B. Turunan manual - 20 poin

Kasus 1 memakai nilai tetap berikut:

$$\begin{aligned} \mathbf{x} &= \begin{bmatrix} 2 & -1 \end{bmatrix}, \quad \mathbf{W}^{(1)} = \begin{bmatrix} 0.5 & -0.5 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 & 0 \end{bmatrix}, \\ \mathbf{W}^{(2)} &= \begin{bmatrix} 2 & -1 \end{bmatrix}, \quad \mathbf{b}^{(2)} = 0.5, \quad y = 1 \end{aligned}$$

Tuliskan penurunan Anda **berurutan** di sini, satu baris satu langkah:

- $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} = p - y = \text{sigma}(2.5) - 1 \approx -0.0758582$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}} \approx \begin{bmatrix} -0.1137873 & -0.0758582 \end{bmatrix}$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{b}^{(2)}} \approx -0.0758582$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{h}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{h}} \approx \begin{bmatrix} -0.1517164 & 0.0758582 \end{bmatrix}$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{h}} \odot \text{ReLU}'(\mathbf{z}^{(1)}) \approx \begin{bmatrix} -0.1517164 & 0.0758582 \end{bmatrix}$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} \right)^T \mathbf{x} \approx \begin{bmatrix} -0.3034328 & 0.1517164 \end{bmatrix}$

$0.1517164 \ \& \ -0.0758582 \ \end{bmatrix}$

$$7. \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}} \approx \begin{bmatrix} -0.1517164 \ \& \ 0.0758582 \end{bmatrix}$$

Cantumkan pula shape setiap gradien:

- $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(2)}}$: $(1,)$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}}$: $(1,2)$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(2)}}$: $(1,)$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{h}}$: $(2,)$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(1)}}$: $(2,)$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}$: $(2,2)$
- $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}}$: $(2,)$

```
In [2]: x = np.array([2.0, -1.0])
W1 = np.array([[0.5, -0.5], [1.0, 1.0]])
b1 = np.array([0.0, 0.0])
W2 = np.array([2.0, -1.0])
b2 = 0.5
y = 1.0

def forward(x, W1, b1, W2, b2, y):
    """TODO 1: kembalikan dict berisi z1, h, z2, p, dan loss."""
    z1 = W1 @ x + b1
    h = np.maximum(0, z1)
    z2 = W2 @ h + b2
    p = 1 / (1 + np.exp(-z2))
    loss = -(y * np.log(p) + (1 - y) * np.log(1 - p))

    return {
        'z1': z1,
        'h': h,
        'z2': z2,
        'p': p,
        'loss': loss
    }

nilai = forward(x, W1, b1, W2, b2, y)
print({k: np.round(v, 6) for k, v in nilai.items()})

# Pemeriksaan wajib: jangan diubah.
assert np.allclose(nilai['z1'], [1.5, 1.0]), 'z1 belum benar'
assert np.isclose(nilai['z2'], 2.5), 'logit belum benar'
assert np.isclose(nilai['loss'], 0.0788897, atol=1e-6), 'loss belum benar'
print('forward pass sesuai Kasus 1')

{'z1': array([1.5, 1. ]), 'h': array([1.5, 1. ]), 'z2': np.float64(2.5), 'p': np.float64(0.924142), 'loss': np.float64(0.078889)}
forward pass sesuai Kasus 1
```

```
In [3]: def backward(x, W1, W2, y, nilai):
    """TODO 2: kembalikan dict gradien untuk 'W1', 'b1', 'W2', 'b2'.
```

Urutan pengerjaan: dz2 -> (dW2, db2, dh) -> dz1 -> (dW1, db1).
Ingat gradien lokal ReLU dan bentuk perkalian luar untuk dW1.
"""

```
dz2 = nilai['p'] - y
dW2 = dz2 * nilai['h']
db2 = dz2
dh = dz2 * W2
dz1 = dh * (nilai['z1'] > 0)
dW1 = np.outer(dz1, x)
db1 = dz1
```

```
return {
    'W1': dW1,
    'b1': db1,
    'W2': dW2,
    'b2': db2
}
```

```
grad_manual = backward(x, W1, W2, y, nilai)
```

```
for nama, v in grad_manual.items():
    print(f'{nama:>3}: {np.round(v, 7)}')
```

Pemeriksaan wajib: dua angka kunci dari modul.

```
assert np.allclose(grad_manual['W2'], [-0.1137873, -0.0758582], atol=1e-6)
```

```
assert grad_manual['W1'].shape == W1.shape, 'shape dW1 harus sama dengan W1'
print('gradien manual sesuai angka acuan')
```

```
W1: [[-0.3034327  0.1517164]
```

```
 [ 0.1517164 -0.0758582]]
```

```
b1: [-0.1517164  0.0758582]
```

```
W2: [-0.1137873 -0.0758582]
```

```
b2: -0.0758582
```

gradien manual sesuai angka acuan

C. Autograd - 20 poin

Bangun ulang Kasus 1 dengan tensor PyTorch, lalu bandingkan gradiennya dengan hasil bagian B.

```
In [4]: tW1 = torch.tensor(W1, requires_grad=True)
tb1 = torch.tensor(b1, requires_grad=True)
tW2 = torch.tensor(W2, requires_grad=True)
tb2 = torch.tensor(b2, requires_grad=True)
tx, ty = torch.tensor(x), torch.tensor(y)
kriteria = nn.BCEWithLogitsLoss()

def forward_torch():
    """TODO 3: hitung loss dengan BCEWithLogitsLoss pada LOGIT."""
    z1 = tW1 @ tx + tb1
    h = torch.relu(z1)
    z2 = tW2 @ h + tb2
    loss = kriteria(z2, ty)
    return loss
```

```

loss = forward_torch()
loss.backward()

for nama, t in [('W1', tw1), ('b1', tb1), ('W2', tw2), ('b2', tb2)]:
    selisih = np.max(np.abs(t.grad.numpy() - grad_manual[nama]))
    print(f'{nama}: autograd = {np.round(t.grad.numpy(), 7)}    selisih maks = {selisih}')
    assert selisih < 1e-10, f'gradien {nama} belum cocok dengan hasil manual'
print('autograd cocok dengan backward manual')

```

```

W1: autograd = [[-0.3034327  0.1517164]
 [ 0.1517164 -0.0758582]]    selisih maks = 0.00e+00
b1: autograd = [-0.1517164  0.0758582]    selisih maks = 0.00e+00
W2: autograd = [-0.1137873 -0.0758582]    selisih maks = 0.00e+00
b2: autograd = -0.0758582    selisih maks = 0.00e+00
autograd cocok dengan backward manual

```

```

In [5]: # TODO 4: panggil backward() sekali lagi TANPA menghapus gradien,
#         cetak tw2.grad, lalu hapus gradien dan hitung ulang.
#         Jelaskan hasilnya pada sel markdown di bawah.

loss = forward_torch()
loss.backward(retain_graph=True)

loss.backward()
print("tw2.grad setelah backward kedua:", tw2.grad)

tw1.grad = None
tb1.grad = None
tw2.grad = None
tb2.grad = None

loss = forward_torch()
loss.backward()

print("tw2.grad setelah gradien dihapus dan backward ulang:", tw2.grad)

```

```

tw2.grad setelah backward kedua: tensor([-0.3414, -0.2276])
tw2.grad setelah gradien dihapus dan backward ulang: tensor([-0.1138, -0.0759])

```

Penjelasan akumulasi gradien:

Jika `backward()` dipanggil dua kali tanpa menghapus gradien, nilai `.grad` menjadi **2 kali lipat** karena gradien baru ditambahkan ke gradien sebelumnya.

Dalam training loop, `optimizer.zero_grad()` digunakan untuk menghapus gradien dari iterasi sebelumnya sebelum melakukan `backward()` pada batch berikutnya, sehingga gradien tidak terus terakumulasi antar-iterasi.

D. Gradient checking - 20 poin

Bandingkan gradien analitik dengan selisih terpusat:

$$g_{\text{num}} = \frac{\mathcal{L}(\theta + \epsilon) - \mathcal{L}(\theta - \epsilon)}{2\epsilon},$$

$$\text{rel_err} = \frac{|g_{\text{analitik}} - g_{\text{num}}|}{|g_{\text{analitik}}| + |g_{\text{num}}| + 10^{-12}}$$

Ambang lulus: seluruh baris di bawah 10^{-5} .

```
In [6]: EPS = 1e-5

def loss_dengan(param, i, delta):
    parameter = {
        'W1': np.copy(w1),
        'b1': np.copy(b1),
        'W2': np.copy(w2),
        'b2': np.copy(b2)
    }

    parameter[param][i] += delta

    nilai = forward(
        x,
        parameter['W1'],
        parameter['b1'],
        parameter['W2'],
        parameter['b2'],
        y
    )

    return nilai['loss']

def finite_difference(param, i):
    """TODO 6: kembalikan gradien numerik dengan selisih terpusat."""
    loss_plus = loss_dengan(param, i, EPS)
    loss_minus = loss_dengan(param, i, -EPS)

    return (loss_plus - loss_minus) / (2 * EPS)

indeks = ([('W1', (0, 0)), ('W1', (0, 1)), ('W1', (1, 0)), ('W1', (1, 1))]
          + [('b1', (0,)), ('b1', (1,))]
          + [('W2', (0,)), ('W2', (1,))]
          + [('b2', ())])

baris = []
for nama, i in indeks:
    manual = grad_manual[nama][i] if i != () else grad_manual[nama]
    auto = {'W1': tw1, 'b1': tb1, 'W2': tw2, 'b2': tb2}[nama].grad.numpy()
    auto = auto[i] if i != () else auto
    numerik = finite_difference(nama, i)
    rel = abs(manual - numerik) / (abs(manual) + abs(numerik) + 1e-12)
    baris.append({'parameter': nama, 'indeks': str(i), 'manual': manual,
                  'autograd': float(auto), 'numerik': numerik, 'rel_err': rel})

tabel = pd.DataFrame(baris)
print(tabel.to_string(index=False))
print('\nrelative error maksimum:', tabel['rel_err'].max())
```

```
assert len(tabel) == 9, 'tabel harus memuat sembilan komponen parameter'
assert tabel['rel_err'].max() < 1e-5, 'masih ada baris yang melampaui ambang'
```

parameter	indeks	manual	autograd	numerik	rel_err
W1	(0, 0)	-0.30343272	-0.30343272	-0.30343272	1.00453007e-10
W1	(0, 1)	0.15171636	0.15171636	0.15171636	2.95622632e-11
W1	(1, 0)	0.15171636	0.15171636	0.15171636	2.95622632e-11
W1	(1, 1)	-0.07585818	-0.07585818	-0.07585818	5.73360674e-11
b1	(0,)	-0.15171636	-0.15171636	-0.15171636	2.95622632e-11
b1	(1,)	0.07585818	0.07585818	0.07585818	5.73360674e-11
W2	(0,)	-0.11378727	-0.11378727	-0.11378727	6.76755052e-11
W2	(1,)	-0.07585818	-0.07585818	-0.07585818	5.73360674e-11
b2	(,)	-0.07585818	-0.07585818	-0.07585818	5.73360674e-11

relative error maksimum: 1.004530066507852e-10

```
In [7]: # TODO 7: simpan tabel ke M02_NIM_metrics.csv (ganti NIM dengan NIM Anda).
tabel.insert(0, 'run_id', 'gradcheck')
tabel.insert(1, 'seed', SEED)
tabel.to_csv(f'M02_{NIM}_metrics.csv', index=False)
print('tersimpan')
```

tersimpan

Checkpoint menit ke-95. Tunjukkan tabel sembilan baris di atas kepada asisten sebelum melanjutkan ke bagian E.

E. Diagnosis training loop - 20 poin

Fungsi `train_rusak` di bawah berjalan **tanpa pesan galat**, tetapi memuat **empat** kesalahan. Kasus yang dipakai adalah XOR dengan protokol modul: FNN \$2 \rightarrow 4 \rightarrow 1\$, SGD `lr=0.1`, 400 epoch, satu batch penuh.

```
In [8]: X_xor = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_xor = torch.tensor([[0.0], [1.0], [1.0], [0.0]])

def train_rusak(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)
    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.Linear(4, 1),
    )
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        logits = model(X_xor)
        loss = kriteria(torch.sigmoid(logits), y_xor)
        opt.step()
        loss.backward()
        riwayat.append(loss.item())
    return model, riwayat
```

```
model_rusak, riwayat_rusak = train_rusak()
print(f'loss awal : {riwayat_rusak[0]:.4f}')
print(f'loss akhir : {riwayat_rusak[-1]:.4f}')
with torch.no_grad():
    print('prediksi :', (torch.sigmoid(model_rusak(X_xor)) > 0.5).int().flatten())
    print('target :', y_xor.int().flatten().tolist())
```

```

-----
RuntimeError                                Traceback (most recent call last)
Cell In[8], line 22
    19         riwayat.append(loss.item())
    20     return model, riwayat
--> 22 model_rusak, riwayat_rusak = train_rusak()
    23 print(f'loss awal : {riwayat_rusak[0]:.4f}')
    24 print(f'loss akhir : {riwayat_rusak[-1]:.4f}')

Cell In[8], line 18, in train_rusak(epoch, lr)
    16     loss = kriteria(torch.sigmoid(logits), y_xor)
    17     opt.step()
--> 18     loss.backward()
    19     riwayat.append(loss.item())
    20     return model, riwayat

File ~\AppData\Roaming\Python\Python312\site-packages\torch\tensor.py:626, in Tensor.backward(self, gradient, retain_graph, create_graph, inputs)
    616 if has_torch_function_unary(self):
    617     return handle_torch_function(
    618         Tensor.backward,
    619         (self,),
    (...) 624         inputs=inputs,
    625     )
--> 626 torch.autograd.backward(
    627     self, gradient, retain_graph, create_graph, inputs=inputs
    628 )

File ~\AppData\Roaming\Python\Python312\site-packages\torch\autograd\__init__.py:347, in backward(tensors, grad_tensors, retain_graph, create_graph, grad_variables, inputs)
    342     retain_graph = create_graph
    344 # The reason we repeat the same comment below is that
    345 # some Python versions print out the first line of a multi-line function
    346 # calls in the traceback and some print out the last line
--> 347 _engine_run_backward(
    348     tensors,
    349     grad_tensors_,
    350     retain_graph,
    351     create_graph,
    352     inputs,
    353     allow_unreachable=True,
    354     accumulate_grad=True,
    355 )

File ~\AppData\Roaming\Python\Python312\site-packages\torch\autograd\graph.py:823, in _engine_run_backward(t_outputs, *args, **kwargs)
    821     unregister_hooks = _register_logging_hooks_on_whole_graph(t_outputs)
    822 try:
--> 823     return Variable._execution_engine.run_backward( # Calls into the C++ engine to run the backward pass
    824         t_outputs, *args, **kwargs
    825     ) # Calls into the C++ engine to run the backward pass
    826 finally:
    827     if attach_logging_hooks:

```

RuntimeError: one of the variables needed for gradient computation has been modified by an inplace operation: [torch.DoubleTensor [4, 1]], which is output 0 of AsStrided Backward0, is at version 2; expected version 1 instead. Hint: enable anomaly detection to find the operation that failed to compute its gradient, with `torch.autograd.set_detect_anomaly(True)`.

Temuan kesalahan

No	Baris kode bermasalah	Mengapa keliru	Gejala yang terlihat
1	<code>opt.step()</code> sebelum <code>loss.backward()</code>	<code>optimizer.step()</code> harus dilakukan setelah gradien dihitung oleh <code>backward()</code> . Jika dibalik, parameter diperbarui sebelum memiliki gradien dari loss saat ini.	Parameter tidak diperbarui berdasarkan gradien iterasi tersebut dan training tidak berjalan sebagaimana mestinya.
2	Tidak ada <code>opt.zero_grad()</code> di awal setiap iterasi	Gradien pada <code>.grad</code> di PyTorch diakumulasikan secara default. Gradien dari iterasi sebelumnya harus dihapus sebelum menghitung gradien baru.	Gradien menumpuk dari iterasi ke iterasi sehingga update parameter menjadi tidak sesuai.
3	<code>loss = kriteria(torch.sigmoid(logits), y_xor)</code>	<code>BCEWithLogitsLoss</code> sudah menggabungkan operasi sigmoid dengan binary cross-entropy, sehingga inputnya harus berupa logits mentah, bukan hasil sigmoid.	Perhitungan loss tidak sesuai dengan penggunaan <code>BCEWithLogitsLoss</code> dan pembelajaran menjadi tidak optimal.
4	<code>nn.Linear(2, 4), nn.Linear(4, 1)</code> tanpa fungsi aktivasi di antaranya	Dua layer linear tanpa aktivasi non-linear tetap ekuivalen dengan satu transformasi linear, sehingga model tidak dapat membentuk batas keputusan non-linear yang diperlukan untuk XOR.	Model tidak mampu mempelajari pola XOR dengan benar; loss sulit turun hingga rendah dan prediksi tidak seluruhnya sesuai target.

```
In [9]: # TODO 8: perbaiki SATU PER SATU. Salin train_rusak, perbaiki satu kesalahan,
#         jalankan, lalu catat loss akhirnya. Ulangi sampai keempatnya beres.
#         Simpan setiap tahap ke daftar berikut.
```

```
tahap = []
```

```
# PERBAIKAN 1
```

```
# Perbaiki urutan backward() dan optimizer.step()
```

```

def train_tahap_1(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)

    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.Linear(4, 1),
    )

    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        logits = model(X_xor)
        loss = kriteria(torch.sigmoid(logits), y_xor)

        loss.backward()
        opt.step()

        riwayat.append(loss.item())

    return model, riwayat

model, riwayat = train_tahap_1()

with torch.no_grad():
    prediksi = (torch.sigmoid(model(X_xor)) > 0.5).int().flatten()
    benar = int((prediksi == y_xor.flatten().int()).sum().item())

tahap.append({
    'tahap': 'perbaikan-1',
    'yang_diperbaiki': 'loss.backward() dilakukan sebelum optimizer.step()',
    'loss_awal': riwayat[0],
    'loss_akhir': riwayat[-1],
    'benar': benar
})

# PERBAIKAN 2
# Tambahkan optimizer.zero_grad()
def train_tahap_2(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)

    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.Linear(4, 1),
    )

    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        opt.zero_grad()

        logits = model(X_xor)

```

```

        loss = kriteria(torch.sigmoid(logits), y_xor)

        loss.backward()
        opt.step()

        riwayat.append(loss.item())

    return model, riwayat

model, riwayat = train_tahap_2()

with torch.no_grad():
    prediksi = (torch.sigmoid(model(X_xor)) > 0.5).int().flatten()
    benar = int((prediksi == y_xor.flatten().int()).sum().item())

tahap.append({
    'tahap': 'perbaikan-2',
    'yang_diperbaiki': 'optimizer.zero_grad() ditambahkan',
    'loss_awal': riwayat[0],
    'loss_akhir': riwayat[-1],
    'benar': benar
})

# PERBAIKAN 3
# BCEWithLogitsLoss menerima logits langsung
def train_tahap_3(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)

    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.Linear(4, 1),
    )

    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        opt.zero_grad()

        logits = model(X_xor)
        loss = kriteria(logits, y_xor)

        loss.backward()
        opt.step()

        riwayat.append(loss.item())

    return model, riwayat

model, riwayat = train_tahap_3()

with torch.no_grad():
    prediksi = (torch.sigmoid(model(X_xor)) > 0.5).int().flatten()

```

```

        benar = int((prediksi == y_xor.flatten().int()).sum().item())

tahap.append({
    'tahap': 'perbaikan-3',
    'yang_diperbaiki': 'BCEWithLogitsLoss menerima logits langsung tanpa sigmoid',
    'loss_awal': riwayat[0],
    'loss_akhir': riwayat[-1],
    'benar': benar
})

# PERBAIKAN 4
# Tambahkan aktivasi non-linear ReLU di antara layer
def train_tahap_4(epoch: int = 400, lr: float = 0.1):
    seed_everything(SEED)

    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.ReLU(),
        nn.Linear(4, 1),
    )

    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        opt.zero_grad()

        logits = model(X_xor)
        loss = kriteria(logits, y_xor)

        loss.backward()
        opt.step()

        riwayat.append(loss.item())

    return model, riwayat

model, riwayat = train_tahap_4()

with torch.no_grad():
    prediksi = (torch.sigmoid(model(X_xor)) > 0.5).int().flatten()
    benar = int((prediksi == y_xor.flatten().int()).sum().item())

tahap.append({
    'tahap': 'perbaikan-4',
    'yang_diperbaiki': 'menambahkan ReLU di antara kedua layer Linear',
    'loss_awal': riwayat[0],
    'loss_akhir': riwayat[-1],
    'benar': benar
})

pd.DataFrame(tahap)

```

Out[9]:

	tahap	yang_diperbaiki	loss_awal	loss_akhir	benar
0	perbaikan-1	loss.backward() dilakukan sebelum optimizer.st...	0.73089302	0.69314718	2
1	perbaikan-2	optimizer.zero_grad() ditambahkan	0.73089302	0.70045426	2
2	perbaikan-3	BCEWithLogitsLoss menerima logits langsung tan...	0.69997429	0.69335681	2
3	perbaikan-4	menambahkan ReLU di antara kedua layer Linear	0.70795106	0.42094491	4

```
In [10]: def train_benar(epoch: int = 400, lr: float = 0.2):
    """TODO 9: versi yang sudah bebas dari keempat kesalahan."""

    seed_everything(SEED)

    model = nn.Sequential(
        nn.Linear(2, 4),
        nn.ReLU(),
        nn.Linear(4, 1),
    )

    opt = torch.optim.SGD(model.parameters(), lr=lr)
    kriteria = nn.BCEWithLogitsLoss()
    riwayat = []

    for _ in range(epoch):
        opt.zero_grad()

        logits = model(X_xor)
        loss = kriteria(logits, y_xor)

        loss.backward()
        opt.step()

        riwayat.append(loss.item())

    return model, riwayat

model_benar, riwayat_benar = train_benar()
print(f'loss akhir: {riwayat_benar[-1]:.4f}')
with torch.no_grad():
    prediksi = (torch.sigmoid(model_benar(X_xor)) > 0.5).int().flatten()
print('prediksi :', prediksi.tolist())

assert riwayat_benar[-1] < 0.1, 'loss akhir harus di bawah 0,1'
assert torch.equal(prediksi, y_xor.int().flatten()), 'keempat titik XOR harus benar'
print('training loop sudah benar')
```

```
loss akhir: 0.0736
prediksi : [0, 1, 1, 0]
training loop sudah benar
```

```
In [11]: # TODO 10: gabungkan catatan tahap perbaikan ke metrics.csv.
df_tahap = pd.DataFrame(tahap)
df_tahap.insert(0, 'seed', SEED)
df_tahap.to_csv(f'M02_{NIM}_metrics_loop.csv', index=False)
print(df_tahap.to_string(index=False))
```

	seed	tahap	yang diperbaiki	loss_awa
1	loss_akhir benar	24 perbaikan-1	loss.backward() dilakukan sebelum optimizer.step()	0.7308930
2	0.69314718	2	optimizer.zero_grad() ditambahkan	0.7308930
2	0.70045426	2		
		24 perbaikan-3	BCEWithLogitsLoss menerima logits langsung tanpa sigmoid	0.6999742
9	0.69335681	2		
		24 perbaikan-4	menambahkan ReLU di antara kedua layer Linear	0.7079510
6	0.42094491	4		

F. Tugas individu

Kerjakan ketiganya di sel-sel baru di bawah bagian ini.

1. **Perluasan jaringan.** Tambahkan neuron ketiga pada hidden layer: baris $[-1 \setminus 0.5]$ pada $\mathbf{W}^{(1)}$, bias 0.25 , dan komponen -0.5 pada $\mathbf{W}^{(2)}$. Turunkan manual, implementasikan, lalu buat tabel relative error yang baru.
2. **Batch dua contoh.** Tambahkan $\mathbf{x}_2 = [-1 \setminus 3]$ dengan $y_2 = 0$, pakai rata-rata loss, dan jelaskan di langkah mana gradien kedua contoh dijumlahkan.
3. **Laporan diagnosis.** Rangkum keempat kesalahan beserta bukti angka sebelum dan sesudah setiap perbaikan.

```
In [12]: # F1. Perluasan jaringan: 3 neuron pada hidden layer
```

```
W1_3 = np.array([
    [0.5, -0.5],
    [1.0, 1.0],
    [-1.0, 0.5]
])

b1_3 = np.array([0.0, 0.0, 0.25])

W2_3 = np.array([2.0, -1.0, -0.5])
b2_3 = 0.5

nilai_3 = forward(x, W1_3, b1_3, W2_3, b2_3, y)

print("z1 :", nilai_3['z1'])
print("h :", nilai_3['h'])
print("z2 :", nilai_3['z2'])
print("p :", nilai_3['p'])
print("loss:", nilai_3['loss'])
```

```

z1 : [ 1.5  1.  -2.25]
h  : [1.5 1.  0. ]
z2 : 2.5
p   : 0.9241418199787566
loss: 0.07888973429254952

```

```

In [13]: grad_3 = backward(
    x,
    w1_3,
    w2_3,
    y,
    nilai_3
)

```

```

print("dw1:")
print(grad_3['w1'])

```

```

print("\ndb1:")
print(grad_3['b1'])

```

```

print("\ndw2:")
print(grad_3['w2'])

```

```

print("\ndb2:")
print(grad_3['b2'])

```

```

dw1:
[[-0.30343272  0.15171636]
 [ 0.15171636 -0.07585818]
 [ 0.          -0.          ]]

```

```

db1:
[-0.15171636  0.07585818  0.          ]

```

```

dw2:
[-0.11378727 -0.07585818 -0.          ]

```

```

db2:
-0.07585818002124345

```

```

In [14]: EPS_3 = 1e-5

```

```

def loss_dengan_3(param, i, delta):
    parameter = {
        'w1': np.copy(w1_3),
        'b1': np.copy(b1_3),
        'w2': np.copy(w2_3),
        'b2': np.copy(b2_3)
    }

    parameter[param][i] += delta

    nilai = forward(
        x,
        parameter['w1'],
        parameter['b1'],

```

```

        parameter['W2'],
        parameter['b2'],
        y
    )

    return nilai['loss']

def finite_difference_3(param, i):
    loss_plus = loss_dengan_3(param, i, EPS_3)
    loss_minus = loss_dengan_3(param, i, -EPS_3)

    return (loss_plus - loss_minus) / (2 * EPS_3)

indeks_3 = (
    [('W1', (0, 0)), ('W1', (0, 1)),
     ('W1', (1, 0)), ('W1', (1, 1)),
     ('W1', (2, 0)), ('W1', (2, 1))]
    + [('b1', (0,)), ('b1', (1,)), ('b1', (2,))]
    + [('W2', (0,)), ('W2', (1,)), ('W2', (2,))]
    + [('b2', (,))]
)

baris_3 = []

for nama, i in indeks_3:
    manual = grad_3[nama][i] if i != () else grad_3[nama]
    numerik = finite_difference_3(nama, i)

    rel = abs(manual - numerik) / (
        abs(manual) + abs(numerik) + 1e-12
    )

    baris_3.append({
        'parameter': nama,
        'indeks': str(i),
        'manual': manual,
        'numerik': numerik,
        'rel_err': rel
    })

tabel_3 = pd.DataFrame(baris_3)

print(tabel_3.to_string(index=False))
print("\nrelative error maksimum:", tabel_3['rel_err'].max())

assert len(tabel_3) == 13
assert tabel_3['rel_err'].max() < 1e-5

```

parameter	indeks	manual	numerik	rel_err
W1	(0, 0)	-0.30343272	-0.30343272	1.00453007e-10
W1	(0, 1)	0.15171636	0.15171636	2.95622632e-11
W1	(1, 0)	0.15171636	0.15171636	2.95622632e-11
W1	(1, 1)	-0.07585818	-0.07585818	5.73360674e-11
W1	(2, 0)	0.00000000	0.00000000	0.00000000e+00
W1	(2, 1)	-0.00000000	0.00000000	0.00000000e+00
b1	(0,)	-0.15171636	-0.15171636	2.95622632e-11
b1	(1,)	0.07585818	0.07585818	5.73360674e-11
b1	(2,)	0.00000000	0.00000000	0.00000000e+00
W2	(0,)	-0.11378727	-0.11378727	6.76755052e-11
W2	(1,)	-0.07585818	-0.07585818	5.73360674e-11
W2	(2,)	-0.00000000	0.00000000	0.00000000e+00
b2	()	-0.07585818	-0.07585818	5.73360674e-11

relative error maksimum: 1.004530066507852e-10

In [15]: *# F2. Batch dua contoh*

```
x_batch = np.array([
    [2.0, -1.0],
    [-1.0, 3.0]
])

y_batch = np.array([1.0, 0.0])

print("x_batch:")
print(x_batch)

print("\ny_batch:")
print(y_batch)
```

x_batch:
[[2. -1.]
 [-1. 3.]]

y_batch:
[1. 0.]

In [16]: *# F2. Loss untuk masing-masing contoh dan mean loss*

```
losses_batch = []

for xb, yb in zip(x_batch, y_batch):
    nilai_b = forward(
        xb,
        W1,
        b1,
        W2,
        b2,
        yb
    )

    losses_batch.append(nilai_b['loss'])

loss_batch = np.mean(losses_batch)
```

```
print("loss contoh 1:", losses_batch[0])
print("loss contoh 2:", losses_batch[1])
print("mean loss      :", loss_batch)
```

```
loss contoh 1: 0.07888973429254952
loss contoh 2: 0.2014132779827524
mean loss      : 0.14015150613765096
```

In [17]: *# F2. Gradien masing-masing contoh*

```
nilai_1 = forward(x_batch[0], W1, b1, W2, b2, y_batch[0])
nilai_2 = forward(x_batch[1], W1, b1, W2, b2, y_batch[1])

grad_1 = backward(
    x_batch[0],
    W1,
    W2,
    y_batch[0],
    nilai_1
)

grad_2 = backward(
    x_batch[1],
    W1,
    W2,
    y_batch[1],
    nilai_2
)

# Rata-rata gradien untuk batch
grad_batch = {}

for nama in ['W1', 'b1', 'W2', 'b2']:
    grad_batch[nama] = (grad_1[nama] + grad_2[nama]) / 2

print("Gradien batch:")
for nama in grad_batch:
    print(f"\n{nama}:")
    print(grad_batch[nama])
```

Gradien batch:

```
W1:
[[-0.15171636  0.07585818]
 [ 0.16707094 -0.31156738]]

b1:
[-0.07585818 -0.05328367]

W2:
[-0.05689364  0.14449643]

b2:
0.05328367189255645
```

G. Pertanyaan analisis

1. Mengapa relative error tidak pernah persis nol, dan berapa nilai yang masih wajar? TODO
2. Apa yang terjadi pada tabel bila $\epsilon = 10^{-9}$? Jalankan dan jelaskan. TODO
3. Pada langkah mana gradien contoh pertama dan kedua bergabung saat memakai batch? TODO
4. Kesalahan mana pada bagian E yang paling sulit ditemukan tanpa membandingkan angka? TODO
5. Apa beda peran backpropagation dan optimizer? (maksimal tiga kalimat) TODO

Checklist sebelum mengumpulkan

- ☐ Identitas, seed, versi library, dan device tercantum.
- ☐ Seluruh `TODO` dan `raise NotImplementedError` sudah diganti.
- ☐ Turunan manual ditulis pada sel markdown bagian B.
- ☐ Tabel relative error memuat sembilan baris dan seluruhnya lulus ambang.
- ☐ Keempat kesalahan bagian E ditemukan, dibuktikan, dan diperbaiki bertahap.
- ☐ Notebook lolos *Restart Kernel and Run All*.
- ☐ Berkas: `M02_NIM.ipynb` , `M02_NIM.pdf` , `M02_NIM_metrics.csv` .